

Tutorial: Despliegue de una Aplicación Web Básica en Elixir



elixir

Asignatura:

UT2-A1 – Despliegue de Aplicaciones Web

Realizado por:

→ *Braulio Herrera, Oliver Rodriguez, Javier Paredes.*

Curso:

2º ASIRM

Fecha:

20 de octubre de 2025

Introducción

Este tutorial guía paso a paso cómo crear, desplegar y mantener una aplicación web básica en **Elixir**, usando **Plug** y **Cowboy** en **Ubuntu Server 24.04**.

La aplicación mostrará información dinámica del cliente (IP, navegador, resolución de pantalla, hora, versión de Elixir, etc.)

Además, se documentan los errores encontrados durante el proceso y cómo solucionarlos.

♦ Características del lenguaje Elixir

- **Tipo:** Lenguaje funcional concurrente, interpretado sobre la máquina virtual **BEAM** (de Erlang).
 - **Modelo de ejecución:** Basado en actores (procesos ligeros).
 - **Uso en la web:** Muy popular junto con el framework **Phoenix**, aunque también puede servir webs directamente usando **Plug y Cowboy**.
 - **Ventajas:** Escalabilidad, tolerancia a fallos y rendimiento alto.
 - **Compilación:** Bytecode ejecutado por BEAM (no compilado a binario nativo).
-

♦ Entorno de trabajo

Sistema: Ubuntu Server 24.04

Elixir: v1.14.0

Erlang/OTP: 25

Editor remoto: Visual Studio Code con extensión *Remote - SSH* (jeanp413)

♦ 1. Instalación de dependencias

Actualizamos los paquetes e instalamos Erlang y Elixir:

```
sudo apt update && sudo apt upgrade -y  
sudo apt install -y erlang elixir git curl
```

◆ 2. Creación del proyecto

Creamos la carpeta de trabajo y un nuevo proyecto:

```
mix new webapp
cd webapp
```

◆ 3. Modificación del archivo `mix.exs`

Editamos `mix.exs` para incluir las dependencias necesarias:

```
defp deps do
  [
    {:plug_cowboy, "~> 2.7"},
    {:plug, "~> 1.18"},
    {:mime, "~> 2.0"}
  ]
end
```

Después ejecutamos:

```
mix deps.get
```

◆ 4. Creación del router

Creamos el archivo `lib/webapp/router.ex`:

```
defmodule Webapp.Router do
  use Plug.Router
  use Plug.ErrorHandler

  plug(:match)
  plug(:dispatch)

  get "/" do
    conn
    |> put_resp_content_type("text/html")
    |> send_resp(200, Webapp.Page.render(conn))
  end
end
```

```
end

match _ do
  send_resp(conn, 404, "Página no encontrada")
end
end
```

◆ 5. Modificación de `lib/webapp/application.ex`

Reemplazamos el contenido con:

```
defmodule Webapp.Application do
  use Application

  @impl true
  def start(_type, _args) do
    children = [
      {Plug.Cowboy, scheme: :http, plug: Webapp.Router, options:
[port: 8080]}
    ]

    opts = [strategy: :one_for_one, name: Webapp.Supervisor]
    Supervisor.start_link(children, opts)
  end
end
```

◆ 6. Creación de la vista

Creamos `lib/webapp/page.ex` con código dinámico:

```
defmodule Webapp.Page do
  def render(conn) do
    ip = Tuple.to_list(conn.remote_ip) |> Enum.join(".")
    ua = get_req_header(conn, "user-agent") |> List.first()
    now = DateTime.utc_now() |> DateTime.to_string()
    version = System.version()
    screen_info = "<script>document.write(screen.width + 'x' +
screen.height)</script>"
  end
end
```

```

"""
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8" />
  <title>Información del Cliente</title>
  <style>
    body { font-family: Arial, sans-serif; background: #f4f7fb;
margin: 0; padding: 20px; }
    h1 { color: #0056b3; text-align: center; }
    .section { background: #eaf3ff; padding: 20px; border-left:
5px solid #007bff; border-radius: 8px; margin-bottom: 20px; }
    .label { font-weight: bold; }
    .container { max-width: 800px; margin: auto; }
  </style>
</head>
<body>
  <div class="container">
    <h1>🌐 Información del Cliente</h1>

    <div class="section">
      <h2>Datos del Servidor</h2>
      <p><span class="label">Fecha y hora:</span> #{now}</p>
      <p><span class="label">Versión de Elixir:</span>
#{:erlang.system_info(:system_version) |> to_string() |>
String.replace("\n", "<br>")}</p>
    </div>

    <div class="section">
      <h2>Datos del Cliente</h2>
      <p><span class="label">IP:</span> #{ip}</p>
      <p><span class="label">Navegador:</span> #{ua}</p>
      <p><span class="label">Resolución de pantalla:</span>
#{screen_info}</p>
    </div>
  </div>
</body>
</html>
"""
end

```

end

◆ 7. Ejecución del servidor HTTP

Ejecutamos:

```
mix run --no-halt
```

Abrimos el navegador y vamos a:

👉 `http://<IP_DEL_SERVIDOR>:8080`

La página mostrará la información del cliente y del servidor correctamente.

◆ 8. Migración a HTTPS con mkcert

8.1 Instalar mkcert

```
sudo apt install mkcert libnss3-tools
```

8.2 Crear autoridad local

```
mkcert -install
```

8.3 Generar certificados para el servidor

```
mkcert 10.2.146.92
```

Esto genera dos archivos:

```
10.2.146.92.pem
```

```
10.2.146.92-key.pem
```

Muévelos dentro del proyecto, por ejemplo a `priv/cert/`.

◆ 9. Configurar HTTPS en Elixir

Edita `lib/webapp/application.ex`:

```
defmodule Webapp.Application do
  use Application

  @impl true
  def start(_type, _args) do
    children = [
      {
        Plug.Cowboy,
        scheme: :https,
        plug: Webapp.Router,
        options: [
          port: 8443,
          otp_app: :webapp,
          keyfile: "priv/cert/10.2.146.92-key.pem",
          certfile: "priv/cert/10.2.146.92.pem"
        ]
      }
    ]

    opts = [strategy: :one_for_one, name: Webapp.Supervisor]
    Supervisor.start_link(children, opts)
  end
end
```

◆ 10. Solución de errores comunes

✗ Error:

Could not start Cowboy adapter, the `:otp_app` option is required when setting relative SSL certfiles

✓ Solución:

Añadir la opción `otp_app: :webapp` en la configuración SSL.

✖ Error:

NET::ERR_CERT_AUTHORITY_INVALID

✔ Solución:

Es normal cuando se usa un certificado autofirmado.

Se puede continuar haciendo clic en “**Avanzado > Aceptar el riesgo / Continuar**”.

El certificado es válido localmente.

◆ 11. Arranque automático con systemd

Creamos el archivo `/etc/systemd/system/webapp.service`:

```
[Unit]
Description=Webapp Elixir Service
After=network.target

[Service]
User=isard
WorkingDirectory=/home/isard/webapp
ExecStart=/usr/bin/mix run --no-halt
Restart=always

[Install]
WantedBy=multi-user.target
```

Habilitamos y arrancamos el servicio:

```
sudo systemctl enable webapp
sudo systemctl start webapp
```

◆ 12. Cambio de puerto del servidor

Si queremos que la aplicación escuche en otro puerto (por ejemplo, **9090** en lugar de 8080), basta con modificar el archivo:

`lib/webapp/application.ex`

Busca esta línea dentro de la lista `children`:

```
{Plug.Cowboy, scheme: :http, plug: Webapp.Router, options: [port: 8080]}
```

y cámbiala por:

```
{Plug.Cowboy, scheme: :http, plug: Webapp.Router, options: [port: 9090]}
```

Guarda los cambios y reinicia el servidor:

```
mix run --no-halt
```

Ahora podrás acceder a:

👉 `http://<IP_DEL_SERVIDOR>:9090`

◆ 13. Cambio de puerto del servidor

Si queremos que la aplicación escuche en otro puerto (por ejemplo, **9090** en lugar de 8080), basta con modificar el archivo:

```
lib/webapp/application.ex
```

Busca esta línea dentro de la lista `children`:

```
{Plug.Cowboy, scheme: :http, plug: Webapp.Router, options: [port: 8080]}
```

y cámbiala por:

```
{Plug.Cowboy, scheme: :http, plug: Webapp.Router, options: [port: 9090]}
```

Guarda los cambios y reinicia el servidor:

```
mix run --no-halt
```

Ahora podrás acceder a:

👉 `http://<IP_DEL_SERVIDOR>:9090`

◆ 14. Añadir formulario de contacto y enlace desde la página principal

Se realizaron varias mejoras en la aplicación para añadir una nueva página de contacto accesible desde la página principal.

1. Enlace al formulario de contacto

En la página principal (definida en el router), se agregó un enlace con estilo visual para dirigir al usuario al formulario de contacto.

Este enlace aparece en la parte superior del contenido principal y redirige a la ruta `/contacto` dentro de la misma pestaña, sin abrir una nueva ventana.

2. Creación de la nueva ruta `/contacto`

En el router (`lib/webapp/router.ex`), se añadió una nueva ruta que muestra la vista del formulario de contacto.

Esta ruta usa un nuevo módulo encargado de renderizar la página.

3. Creación del archivo `lib/webapp/contact.ex`

Se creó un nuevo archivo dentro de la carpeta `lib/webapp/` llamado `contact.ex`, que contiene el HTML del formulario de contacto.

La página incluye los siguientes elementos:

- Campos para **nombre**, **correo Gmail** y **mensaje**.
- Un botón “Enviar” (solo decorativo, sin funcionalidad real).
- Un botón “← Volver a inicio” que permite regresar fácilmente a la página principal.

La interfaz mantiene el mismo estilo visual que el resto de la aplicación, con fondo claro, diseño centrado y botones en color azul.

◆ 15. Resultado final

Accediendo a

👉 <https://10.2.146.92:8443>

La página muestra:

- Fecha y hora actual
- Versión de Elixir y Erlang
- IP y navegador del cliente
- Resolución de pantalla
- Conexión **segura HTTPS** con mkcert

The image shows two screenshots of a web application. The top screenshot is titled 'Información del Cliente' and displays server and client data. The bottom screenshot is titled 'Contacto' and shows a form for sending a message.

Información del Cliente

Contacto

Datos del Servidor

Fecha del servidor (UTC): 2025-10-24 07:48:36.070021Z

Versión de Elixir:

Erlang/OTP 25 [erts-13.2.2.5] [source] [64-bit] [smp:4:4] [ds:4:4:10] [async-threads:1] [jit:ns]

Elixir 1.14.0 (compiled with Erlang/OTP 24)

Hora local actual: 8:57:50

Datos del Cliente

IP: 10.0.1.145

Navegador: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36

Contacto

Nombre:

Tu nombre

Correo Gmail:

tuusuario@gmail.com

Mensaje:

Escribe tu mensaje...

Enviar

← Volver a inicio

